

## Homework 2: Training a Neural Network CPE 487/587

Instructor: Rahul Bhadani

Due: Feb 09, 2026, 11:59 PM  
116 Points

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

### Submission instruction:

Upload a .pdf on Canvas with the format CPE487587-LastFirst-HW-XX. For example, if your name is Sam Wells, your file name should be CPE487587-LastFirst-HW-XX.pdf. If there is a programming assignment, then you should include your source code along with your PDF files in a zip file CPE487587-LastFirst-HW-XX.zip. Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

All CPE 587 students are required to submit their response in a Latex-generated PDF.

Provide a PDF with a solution to your theory portion and github commit hash and GitHub repo URL for your practice portion. In addition, include any plots in the PDF for practice portion if asked.

## Theory

### 1 Vector Sum Reduction of Vector with a Scalar Multiplier (10 Points)

Consider  $y = \sum_{i=1}^n f_i(\mathbf{x}, z)$  where  $f_i(\mathbf{x}, z) = x_i z$  and  $z$  is a scalar.  $\mathbf{x}$  has  $n$  elements.

1. Compute the gradient  $\nabla_{\mathbf{x}} y$ . What's its dimension? **(4 + 1) Points**
2. Compute the gradient  $\nabla_z y$ . What's its dimension? **(4 + 1) Points**

### 2 Chain Rule For Univariate Functions (5 Points)

Consider  $y = f(x) = \ln(\sin(x^3)^2)$ . Introduce intermediate variables, compute intermediate derivatives and apply chain rule to determine  $\frac{dy}{dx}$  **(5 Points)**

### 3 Vector and Matrix Equivalent (6 Points)

Consider a feature matrix  $X \in \mathbb{R}^{n \times d}$  where  $n$  is the number of samples and  $d$  is number of features.  $X$  can be written in its expanded form as

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \quad (1)$$

1. If we consider  $\mathbf{x}_i$  as a feature vector for the  $i$ -th sample, how would you write  $X$  using  $\mathbf{x}_i$ ? **(2 Points)**
2. Consider one sample equivalent of affine transformation as  $z_i = \mathbf{x}_i^\top \mathbf{w} + b$  where  $b$  is a bias. How the expression changes when you consider all samples at once? **(2 Points)**
3. Consider a loss function as  $C = \frac{1}{n} \sum_i v_i^2$  where  $v_i$  is error due to individual sample  $i$ . What would be its vectorized equivalent if you need to take into account all samples? **(2 Points)**

## 4 Weight Dimension in a Neural Network (5 Points)

Consider the problem assigned in Homework 1.  $W_1$  is a matrix for the given sequence of operations in the diagram. Under what condition,  $W_1$  will become one-dimensional vector. You are free to modify the sequence of operation, or eliminate any sequence of operation, keeping the goal as the classification. **(5 Points)**

## 5 Streamplots (5 Points)

Consider the streamplot given in Figure 1. Based on the streamplot, what is the critical point of the function  $f$ ? Is this represent a maximum or minimum? Explain your claim. **(5 Points)**

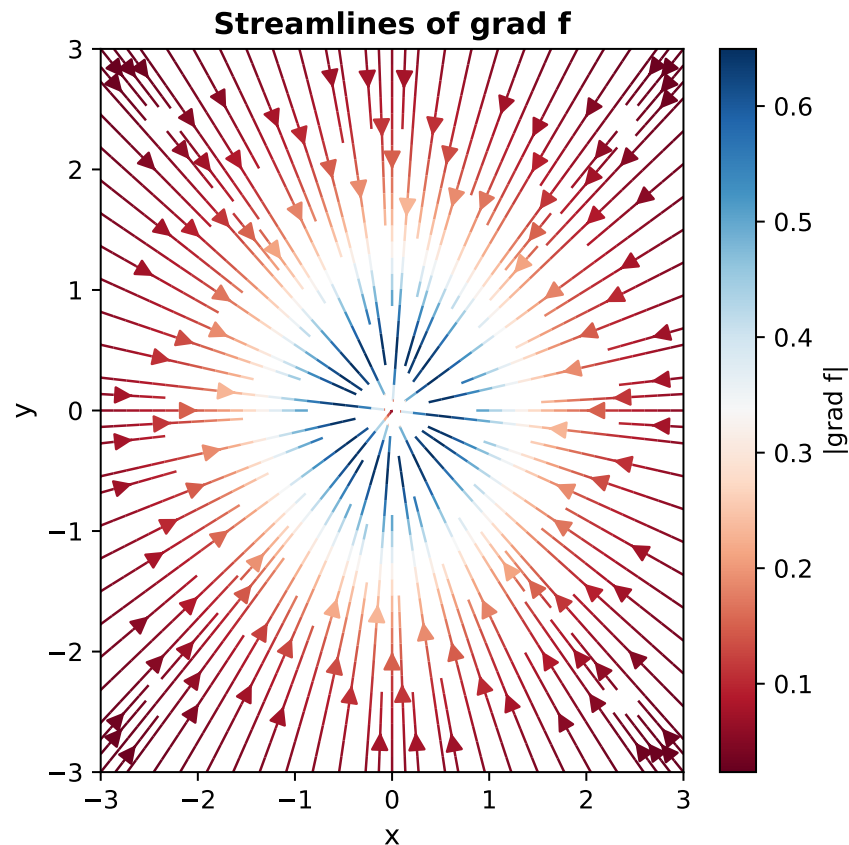


Figure 1: A streamplot

## 6 Gradient Calculations for Multiple Points in Autograd PyTorch (5 Points)

For a bivariate function  $f(x, y)$ , before computing gradients for multiple points via `.backward()`, why do we use `.sum()`? Provide an example to support your answer. **(5 Points)**

### Practice

## 7 Weight Matrix Visualization (35 Points)

Go back to your homework 01. Create a subpackage in your homework Github repo called `animation`.

Add a module called `weight_animation.py` and copy the code from [https://github.com/rahulbhadani/manimations/blob/main/scripts/weight\\_animation.py](https://github.com/rahulbhadani/manimations/blob/main/scripts/weight_animation.py). **(2 Points)**

Add another module called `largewt_animation.py` and copy from [https://github.com/rahulbhadani/manimations/blob/main/scripts/largewt\\_animation.py](https://github.com/rahulbhadani/manimations/blob/main/scripts/largewt_animation.py). **(2 Points)**

`weight_animation.py` provides implementation for a python Class `WeightMatrixAnime` that takes a 3D matrix as an argument and generate animation for matrix visualization. However, this is only fast enough if matrix size is not too large. There is also a function `animate_weight_heatmap` that will actually do the animation.

The usage is below (also provided in `.py` file):

```
1 weight_data = torch.randn(20, 50, 50)
2     animate_weight_heatmap(
3         weight_data,
4         dt=0.1,
5         file_name = "wt_animation",
6         title_str="Neural Weight Distribution"
7     )
```

`largewt_animation.py` provides implementation for a python Class `LargeWeightMatrixAnime` that takes a 3D matrix as an argument and generate animation for matrix visualization. This is suitable for a situation when matrix is large, for example  $1000 \times 1000$ . There is also a function `animate_large_heatmap` that will do the animation.

The usage is below (also provided in `.py` file):

```
1 large_data = torch.randn(20, 1000, 1000)
2     animate_large_heatmap(
3         large_data,
4         dt=0.1,
```

```

5     file_name="large_weights_evolution",
6     title_str="Weight Evolution"
7 )

```

## 7.1 Tasks

1. Modify your `__init__.py` so that `WeightMatrixAnime` and `LargeWeightMatrixAnime` can be imported in your test scripts when necessary. You may also need to import `animate_weight_heatmap` or `animate_large_heatmap` if you decide to only pass weight matrices stacks to create animation. **(2 Points)**
2. Modify `binary_classification` in the file `two_layer_binary_classification.py` of `deepl` subpackage so that you can store intermediate weights from every epoch. **(10 Points)**

A rough code-snippet to accomplish the above task is provided below:

```

1  weights = torch.zeros(steps, size1, size2)
2  for i in range(steps):
3
4      # randomly generate weight matrix but for you
5      # it would come from updated weights
6      # after gradient descent every epochs
7      current = torch.randn(size1, size2)
8      weights[i] = current.clone()
9  return weights

```

3. Why do we need `clone()` function in above code-snippet? **(5 Points)**
4. Create a copy of `binaryclassification_impl.py` called `binaryclassification_animate_impl.py` in `scripts` folder of your homework project directory and modify it to import either `WeightMatrixAnime` or `LargeWeightMatrixAnime` whichever works best for your case. Your task is to access 3D weights matrices returned by `binary_classification` function and create animation. A successful execution of `binaryclassification_animate_impl.py` should create `.mp4` file in `media` folder.

While making commit to GitHub, you should not commit `media` folder. You may add `media/` entry to your `.gitignore` to avoid committing this file inadvertently.

You should be able to generate a four animation movie corresponding to four weights  $W_1 - W_4$  from the model provided in Homework 01. **(10 Points)**

You can also supply custom name for animation movie file as you see fit.

Parameters you would choose for your executing

- (a)  $dt = 0.04$
- (b)  $epochs = 5000$
- (c)  $eta = 0.01$
- (d) number of features = 200
- (e) number of samples = 40000

Now, at this point, you may realize that it takes a long time to finish executing.

5. **BONUS:** you can write a bash script to run the execution in the background. **(5 Points)**
6. Update your README with a section title **HW02Q7** with instructions on how to execute your code, and where to find the .mp4 files generated. **(4 Points)**

## 8 Classification with a Neural Network (45 Points)

Consider a class definition of a Neural Network for a binary classification problem:

```

1  class SimpleNN(nn.Module):
2      def __init__(self, in_features):
3          super(SimpleNN, self).__init__()
4          self.in_features = in_features
5          self.fc1 = nn.Linear(self.in_features, 3)
6          self.fc2 = nn.Linear(3, 4)
7          self.fc3 = nn.Linear(4, 5)
8          self.fc4 = nn.Linear(5, 1)
9          self.relu = nn.ReLU()
10
11     def forward(self, x):
12         x = self.relu(self.fc1(x))
13         x = self.relu(self.fc2(x))
14         x = self.relu(self.fc3(x))
15         x = self.fc4(x)
16         return x

```

1. If your dataset has **m** number of classes present in the ground truth label, what changes you need to make to the above code? Provide your response separately in the accompanied PDF file. **(2 Points)**
2. Given the default values in `nn.Linear` for parameters that are not explicitly set, what are total number of scalar parameters used in the above neural network. You may need to refer to documentation for their API to provide correct answer. **(5 Points)**

3. Download android malware dataset from [https://github.com/rahulbhadani/CPE487587\\_SP26/releases/download/android\\_malware/Android\\_Malware.csv](https://github.com/rahulbhadani/CPE487587_SP26/releases/download/android_malware/Android_Malware.csv). To enable automation of your data analysis, you must download dataset in the specified way as follows:

- (a) Create a shell script called `malwaredatadownload.sh` in your project directory with the following content

```
1  #!/bin/bash
2
3  mkdir -p data
4  cd data
5  wget https://github.com/rahulbhadani/CPE487587_SP26/releases/download/android_malware/Android_Malware.csv
```

Then in the command line, execute the following:

```
1  chmod +x malwaredatadownload.sh
2  ./malwaredatadownload.sh
```

Executing above will directly download the data in your machine. However, you must never commit **data** folder in your GitHub. You, however, do need to commit `malwaredatadownload.sh` file to GitHub. **(5 Points)**

4. Read the dataset using Pandas or Polars library. Dataset contains a number of features related network data packet that helps in classifying what kind of malware that data packet is.

There are following classes of malware:

- (a) Android\_Adware
- (b) Android\_Scareware
- (c) Android\_SMS\_Malware
- (d) Benign

Benign means not a malware.

You should also filter out the following features that are not helpful in classification:

- (a) Flow ID
- (b) Source IP
- (c) Source Port
- (d) Destination IP
- (e) Destination Port
- (f) Protocol
- (g) Timestamp

## 8.1 Deliverables

1. In `deepl` subpackage, add a module `multiclass.py` and add code for `SimpleNN` Class. **(3 Points)**
2. Modify `SimpleNN` neural network class that you will use to create a machine learning model to take number of classes as input parameter as well. **(2 Points)**
3. Create new class in `multiclass.py` called `ClassTrainer`. It will have following parameters that users can specify at run time, and will be set as class variable: **(5 Points)**
  - (a) `X_train`: feature matrix
  - (b) `Y_train`: known label
  - (c) `eta`: learning rate
  - (d) `epoch`: number of epochs
  - (e) `loss`: loss function (e.g. `nn.BCEWithLogitsLoss()`)
  - (f) `optimizer`, for example ADAM (`optim.Adam(model.parameters(), lr=self.eta)`)
  - (g) `loss_vector`: to be set after training, initializer as empty torch Tensors with length of epochs
  - (h) `accuracy_vector`: to be set after training, initializer as empty torch Tensors with length of epochs
  - (i) `model`: a model class (e.g. `SimpleNN` defined above)
  - (j) `device`: cuda device set depending on availability.
4. In addition, `ClassTrainer` will have the following functions: **(15 Points)**
  - (a) `train()`: it would invoke the training process upon calling. This function doesn't take any argument.
  - (b) `test()`: it would perform test using test data. It would take two arguments: `X_test`, and `y_test`.
  - (c) `predict()`: it would make inference using a set of features. It takes only one argument: `X`
  - (d) `save()`: it will save the trained model in onnx format. It may take one argument optionally: file name of the model to be saved.
  - (e) `evaluation()`: this function takes loss vector and accuracy vector and plot to demonstrate the performance on training set. In addition, it also creates a plot of confusion matrix showing correct label and predicted labels, as well as final F1 score, Precision, Recall, and Accuracy for both training and test dataset.

You will need to modify your `__init__.py` accordingly so that you can use them later.



5. In your `scripts` folder, create a file called `multiclass_impl.py` that will be import your classes defined above and perform the following task: **(15 Points)**

- (a) Read data file from the data folder.
  - (b) Perform feature extraction and get rid of above some features I specified above.
  - (c) Perform 80:20 train-test split.
  - (d) instantiate your `SimpleNN` and `Trainer` class.
  - (e) You should specify some default values for data file location, learning rate, epoch, loss function, optimizer, and device. However, write your script in such a way that it should be able to take command line parameters similar to ones specified in [https://github.com/rahulbhadani/cpe487587hw01/blob/main/scripts/binaryclassification\\_impl\\_cmdline.py](https://github.com/rahulbhadani/cpe487587hw01/blob/main/scripts/binaryclassification_impl_cmdline.py).
  - (f) Your `multiclass_impl.py` should also be able to save final accuracy, F1 score, precision and recall for both training and test set in CSV file. The script should also specify a common line parameter for a unique keyword that will be appending to CSV file name containing training/test metrics. Since you are going to train multiple times, CSV file name should also contain timestamp at the time of saving the file.
  - (g) Write a python script `multiclass_eval.py` that takes the same unique keyword specified above as a command line parameter, and look for all CSV files whose name matches that keyword and aggregate all training/test metrics and produce a boxplot of Accuracy, F1 score, precision, and recall for both training and test dataset.
  - (h) Write a shell script `multiclass_impl.sh` that should invoke `multiclass_impl.py` five times with same parameters, including a unique word. For consistency, use "hw02" as the unique keyword. Immediately after invoking `multiclass_impl.py` five times, invoke `multiclass_eval.py` with keyword "hw02" that will aggregate training/test metrics and produce box plot. The boxplot should be saved with a name that should carry the keyword "hw02" and the timestamp.
6. In README, add a section HW02Q8 and provide instructions on how to execute your code to get the boxplot. **(5 Points)**